Necessary commands

```
notepad Dockerfile

mv Dockerfile.txt Dockerfile

cat Dockerfile



git add .

git commit -m "index.html v1 dockerfile added"



git remote add origin https://github.com/divyasai-svg/devops-assignment4.git

git branch -M main

git push origin main

git clone https://github.com/divyasai-svg/devops-assignment4

cd devops-assignment4

ls
```

Docker

```
sudo docker build -t divyamull/devops-assignment4:v1 .

docker push divyamull/devops-assignment4:v1
```

Kubernetes (kubectl)

```
kubectl create deployment devops-assignment-app1 --image=divyamull/devops-assignment4:v1 --port=8080

kubectl expose deployment devops-assignment-app1 --type=NodePort --port=8080 --target-port=80
```

Minikube

```
minikube service devops-assignment-app1
```

```
# Build Docker image

docker build -t divyamull/devops-assignment4:v1 .



# Push image to Docker Hub

docker login

docker push divyamull/devops-assignment4:v1



# Create Kubernetes deployment

kubectl create deployment devops-assignment-app1 --image=divyamull/devops-assignment4:v1 --port=8080



# Expose deployment as a NodePort service

kubectl expose deployment devops-assignment-app1 --type=NodePort --port=8080 --target-port=80



# Open the application in Minikube

minikube service devops-assignment-app1
```

```
# Update image after modifying index.html (if required)
docker build -t divyamull/devops-assignment4:v2 .
docker push divyamull/devops-assignment4:v2
kubectl set image deployment/devops-assignment-app1 \
devops-assignment-app1=divyamull/devops-assignment4:v2

# GitHub commands (using your GitHub username)
git remote add origin https://github.com/divyasai-svg/devops-assignment4.git
git branch -M main
git push -u origin main
```

Give all git commands also

```
# Create project directory
mkdir devops-assignment4
cd devops-assignment4

# Initialize Git repository
git init

# Create index.html
touch index.html
```

```
# Create Dockerfile
touch Dockerfile

# Check repository status
git status

# Add files
git add .

# Commit files
git commit -m "Added index.html and Dockerfile"

# Rename branch to main
git branch -M main

# Add GitHub remote
git remote add origin https://github.com/divyasai-svg/devops-assignment4.git

# Push to GitHub
git push -u origin main
```

```
# Clone the repository (on another system)

git clone https://github.com/divyasai-svg/devops-assignment4.git



# Go to project directory

cd devops-assignment4



# Check files

ls



# Build Docker image

docker build -t divyamull/devops-assignment4:v1 .



# Login to Docker Hub

docker login



# Push image to Docker Hub

docker push divyamull/devops-assignment4:v1

#Run docker image

docker run -d -p 8080:80 --name devops-assignment4 divyamull/devops-assignment4:v1



# Check running containers

docker ps
```

```
# Stop the container
docker stop devops-assignment4

# Remove the container
docker rm devops-assignment4
# Create Kubernetes deployment
kubectl create deployment devops-assignment-app1 --image=divyamull/devops-assignment4:v1 --port=8080

# Expose deployment
kubectl expose deployment devops-assignment-app1 --type=NodePort --port=8080 --target-port=80

# Open service in Minikube
minikube service devops-assignment-app1

# Check deployment
kubectl get deployments

# Check services
kubectl get services

# Check pods
```

```
kubectl get pods
```

Nginx and httpd base code

```
FROM httpd:latest

COPY . /usr/local/apache2/htdocs/
```

```
FROM nginx:latest

COPY . /usr/share/nginx/html/
```

Flask dockerfile base code

```
FROM python:3.10

WORKDIR /app

COPY . /app

EXPOSE 5000

CMD ["python", "app.py"]
```

Jenkins- GitHub script

```
pipeline {
 agent any
 environment {
   DOCKER_CRED = credentials('Docker')
 }
```

```
stages {
 stage('fetch-code') {
  steps {
   git branch: 'main', url: 'https://github.com/divyasai-svg/assign4.git'
  }
 }
 stage('build-image') {
  steps {
   sh 'docker build -t divyamull/add4:${BUILD_NUMBER} .'
  }
 }
 stage('push-image') {
  steps {
   sh ' echo $DOCKER_CRED_PSW | docker login -u $DOCKER_CRED_USR --password-stdin'
   sh 'docker push divyamull/add4:${BUILD_NUMBER}'
  }
 }
 stage('update-minikube') {
  steps {
   sh 'kubectl set image deployment/dd4 add4=divyamull/add4:${BUILD_NUMBER}'
  }
 }
```

```
 }
}
```

To run Kafka

Terminal 1

```
zookeeper-server-start.sh config/zookeeper.properties
```

Terminal 2

```
kafka-server-start.sh config/server.properties
```

Terminal 3

```
kafka-topics.sh --create --topic titanic --bootstrap-server localhost:9092
```

Terminal 4

Run producer and consumer codes

Spark context

Intialization

```
import os
import sys


os.environ["SPARK_HOME"] = "/home/talentum/spark"
os.environ["PYLIB"] = os.environ["SPARK_HOME"] + "/python/lib"
# In below two lines, use /usr/bin/python2.7 if you want to use Python 2
```

```
os.environ["PYSPARK_PYTHON"] = "/usr/bin/python3.6"

os.environ["PYSPARK_DRIVER_PYTHON"] = "/usr/bin/python3"

sys.path.insert(0, os.environ["PYLIB"] +"/py4j-0.10.7-src.zip")

sys.path.insert(0, os.environ["PYLIB"] +"/pyspark.zip")



# NOTE: Whichever package you want mention here.

# os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages com.databricks:spark-xml_2.11:0.6.0 pyspark-shell'

# os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages org.apache.spark:spark-avro_2.11:2.4.0 pyspark-shell'

os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages
com.databricks:spark-xml_2.11:0.6.0,org.apache.spark:spark-avro_2.11:2.4.3 pyspark-shell'

# os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages
com.databricks:spark-xml_2.11:0.6.0,org.apache.spark:spark-avro_2.11:2.4.0 pyspark-shell'




#Entrypoint 2.x

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark SQL basic example").enableHiveSupport().getOrCreate()



# On yarn:

# spark = SparkSession.builder.appName("Spark SQL basic
example").enableHiveSupport().master("yarn").getOrCreate()

# specify .master("yarn")



sc = spark.sparkContext
```

GitHub account config

git config --global user.name "Your Name"

git config --global user.email "your.email@example.com”